



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**AUTOMATED PERSONAL EXPENDITURE MONITORING SYSTEM
DEVELOPED IN C++**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for
the Course of*

DSA0109 – OBJECT ORIENTED PROGRAMMING WITH C++

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

P. Muni Karthik – 192425061

Under the Supervision of

Dr. I. Sudha

SIMATS ENGINEERING

SEPTEMBER -2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Automated Personal Expenditure Monitoring System Developed in C++** ” has been carried out by P. MUNI KARTHIK (192425061), M. SASIDHAR (192425304), KARTHIKEYA (19241109) under the supervision of Dr.I.Sudha and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech – AIML & CSE Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C. Sivasankar

Professor

Department of CSE,

Saveetha School of Engineering

SIMATS, Chennai

COURSE FACULTY

Dr. I. Sudha

Professor

Department of CSE,

Saveetha School of Engineering,

SIMATS, Chennai

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, P. MUNI KARTHIK of the AIML, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Automated Personal Expenditure Monitoring System Developed in C++**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: _____

Signature of Student:

P. Muni Karthik

192425061

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

Student	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx . (%)
M.Sasidhar 192425304	Part 1 – Income & Expenses Management	Income and expense recording;	Testing income entries, transaction operations, data validation	Chapters 1–5, income, expense management, testing and individual reflection	33.3%
P. Muni Karthik / 192425061	Part 2 – Financial Analysis & Expenditure Monitoring	Financial calculations; expenditure aggregation; budget monitoring	Manual verification of totals, balance, savings rate and category sums	Chapters 1–5, financial analysis, testing and individual reflection	33.3%
P. Kartekeya / 192411093	Part 3 – Dashboard, Visualization & System Integration	Streamlit dashboard development; financial data visualization; dashboard integration; transaction display and user interface	Testing dashboard functionality, visualization accuracy, transaction display and system integration	Chapters 1–5, dashboard implementation, testing and individual reflection	33.3%
TOTAL					100%

ABSTRACT

Personal expenditure management is an important practical problem because users need to record income and expenses, understand spending patterns, monitor budgets and evaluate their financial position. Manual calculations can be time-consuming and may lead to arithmetic errors when the number of transactions increases. The Automated Personal Expenditure Monitoring System addresses this problem by maintaining transaction records and automatically producing financial summaries and expenditure analysis.

The core implementation is developed in C++ using structures, vectors, file handling, string streams and formatted input/output. Transactions are stored with date, type, category, description and amount. The financial-analysis component processes the stored records to calculate total income, total expenses, balance, savings rate and category-wise expenses. The completed implementation is also presented through an interactive Personal Finance Dashboard that provides expenditure monitoring, monthly budget information and graphical analytics.

Testing was performed using transaction data entered into the implemented system and the displayed dashboard results. For the recorded sample data, the system calculated total income of ₹25,000.00, total expenses of ₹4,232.00, balance of ₹20,768.00 and savings rate of approximately 83.1%. Category analysis identified Bills as the highest spending category with ₹3,652.00. These outputs were checked against manual calculations to verify correctness.

The system also supports expenditure monitoring through monthly budget information. A predefined monthly budget can be compared with the amount already spent to determine the remaining available budget. The dashboard presents this information through indicators that make it easier to monitor whether expenditure is within the planned limit. This provides a practical connection between transaction-level data processing and personal financial decision-making.

Keywords: Personal Finance; Expenditure Monitoring; Financial Analysis; C++; Transaction Management; Budget Monitoring; Category-wise Expenses

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	3
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	5
4	CHAPTER 4: Implementation, Testing & Results, Project Management	10
5	CHAPTER 5: Conclusion & Reflection	17
	References	18
	Appendices	19

LIST OF TABLES

Table No.	Table Name
Table 1	Individual Contribution Statement
Table 2	Project Objectives
Table 3	Stakeholder and User Requirements
Table 4	Functional and Non-Functional Requirements
Table 5	Design Specifications
Table 6	Alternative Solution Evaluation
Table 7	Trade-off Analysis
Table 8	Sustainability, Ethics, Safety and Security Considerations
Table 9	Risk Register
Table 10	Software Tools and Resources
Table 11	Test Plan and Verification
Table 12	Objective Achievement
Table 13	Project Milestones
Table 14	Team Responsibilities
Table 15	Project Budget

LIST OF FIGURES

Figure No.	Figure Name
Figure 1	System architecture and financial-analysis module flow
Figure 2	Personal Finance Dashboard showing financial summary and budget monitoring
Figure 3	Add New Transaction interface
Figure 4	Transaction records and edit interface
Figure 5	Analytics view showing category-wise expenditure

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Term	Description
C++	Programming language used for the core implementation
CSV	Comma-Separated Values file used for transaction storage
UI	User Interface
UX	User Experience
I/O	Input / Output
₹	Indian Rupee currency symbol
%	Percentage used for savings and budget utilization

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Personal finance management involves recording income, tracking expenses, understanding spending categories and maintaining awareness of available balance. When transactions are maintained manually, users must repeatedly calculate totals and compare expenditure against available income or budget. A simple automated system can reduce this effort by storing transaction information and deriving useful financial indicators from the stored data.

1.2 Need for the Project

The project is needed to provide a structured and automated method for personal expenditure monitoring. It reduces repeated manual calculations and provides a consistent method for determining total income, total expenditure, balance, savings rate and category-wise spending. The analytical view also helps users identify high-spending categories and monitor whether expenditure remains within the selected monthly budget.

1.3 Problem Statement

The engineering problem is to develop a reliable expenditure-monitoring component that accepts income and expense transactions, stores them persistently, processes the records correctly and presents meaningful financial indicators. The system must distinguish income from expenses, aggregate amounts accurately, group expenses by category and calculate derived measures such as balance and savings rate without introducing calculation errors.

1.4 Aim

To develop and validate a financial analysis and expenditure monitoring component that automatically converts recorded personal transactions into accurate financial summaries and categorywise expenditure information.

1.5 Objectives

Objective	Measurable outcome
Automated Financial Calculation	Calculate total income, total expenses, balance and savings rate from transaction data.
Expenditure Monitoring	Track spending and compare expenses with the configured monthly budget.
Category-wise Analysis	Aggregate expenses by category and identify high-spending categories.
Reliable Data Processing	Read, process and save transaction records without loss of financial fields.

1.6 Scope and Expected Outcome

The individual contribution covers the financial-analysis and expenditure-monitoring layer of the complete project. It includes processing transaction records, separating income and expense values, calculating financial totals, deriving balance and savings rate, aggregating expenses by category, monitoring budget utilization and interpreting analytical results. User-interface development and the broader project integration are treated as supporting parts of the complete system.

The expected outcome is a functioning financial-analysis component that produces consistent numerical results from transaction data and presents those results through summary indicators and expenditure analytics. The component is intended for personal or small-scale financial tracking and is not designed as a banking, accounting or investment-management platform.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered common personal finance management approaches, spreadsheet-based expense tracking, manual record keeping and software-based expenditure dashboards. The comparison focused on transaction storage, automatic calculations, category analysis, budget monitoring, ease of use and the level of automation available to an individual user.

2.2 Review of Existing Work

Manual notebooks and spreadsheets are flexible and inexpensive but require users to perform calculations and update summaries themselves. General personal finance applications provide automated summaries and visualizations but may include features beyond the requirements of a lightweight academic project. The proposed system focuses on a smaller and transparent set of financial-analysis operations implemented using fundamental C++ programming concepts and a supporting dashboard.

Existing approach	Advantages	Limitations	Relevance to the system
Manual notebook	Simple; no software required	Manual calculations; difficult aggregation	Motivates automation
Spreadsheet	Flexible; formulas and tables	Requires careful formula maintenance	Provides comparison for automated totals
General finance application	Rich dashboards and features	May be complex for a small academic project	Provides benchmark for usability
Proposed C++ system	Transparent calculations; lightweight; traceable	Limited advanced features	Selected for academic scope and C++ learning

2.3 Engineering Gap

The engineering gap addressed in this project is the need for a simple expenditure-monitoring solution in which transaction data, persistent storage, financial calculations and category-level analysis are connected within one workflow. The solution emphasizes understandable calculations and traceable results rather than complex financial services or advanced predictive analytics.

2.4 Proposed Contribution

My individual contribution addresses the financial-analysis layer of the system. The contribution includes calculation of total income and expenses, balance computation, savings-rate derivation, category-wise expense aggregation and interpretation of expenditure patterns. The calculations are based directly on the stored transaction records, allowing the displayed results to be verified from the underlying data.

The financial-analysis component also supports expenditure monitoring through monthly budget comparison and dashboard indicators. This provides a practical connection between basic C++ data processing and user-oriented financial decision support.

Another important part of the contribution is category-wise expense aggregation. Expense transactions are examined based on their respective categories, such as Food, Bills, Travel and other user-defined categories. The system combines expenses belonging to the same category and calculates the total amount spent under each category. This makes it possible to identify major areas of expenditure and understand the user's spending pattern. The category-wise results can be used to determine the highest spending category and support better expenditure planning.

The financial-analysis component also supports expenditure monitoring through monthly budget comparison. The total amount spent is compared with the predefined monthly budget to determine the amount already utilized and the remaining budget.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement	
Personal user	Record income and expenses and understand financial position.	
System user	View total income, expenses, balance and savings rate.	
Budget-conscious user	Monitor expenditure against a monthly budget.	
Project evaluator	Verify that C++ concepts are applied to a practical problem.	
Requirement	Type	Measurable Target
Transaction recording	Functional	Date, type, category, description and amount are stored.
Income/expense separation	Functional	Income and expenses are accumulated independently.
Financial summary	Functional	Income, expense, balance and savings rate are calculated automatically.
Category analysis	Functional	Expense amounts are aggregated for each category.
Persistent storage	Functional	Transactions are saved and loaded through CSV.
Correctness	Non-Functional	Calculated outputs match manually verified values.
Usability	Non-Functional	Results are presented in readable summaries and dashboard indicators.

3.2 Constraints & Applicable Practices

The financial-analysis component is constrained by the requirement to remain understandable, lightweight and suitable for an Object Oriented Programming with C++ academic project. The implementation uses standard C++ facilities such as structures, vectors, file streams, string streams and formatted output. The stored CSV representation keeps the data simple and portable, but it also limits the system compared with a full database-backed finance application.

3.3 Design Specifications

The design specification for my contribution is based on a transaction-processing pipeline. Each transaction contains date, type, category, description and amount. Financial analysis reads these records, separates income and expenses, calculates aggregate values and groups expense amounts by category. The resulting indicators are then displayed as financial summaries and analytical dashboard elements.

Parameter	Target / Requirement	Unit	Priority
Programming language	C++	—	High
Data storage	CSV file	—	High
Transaction amount	Non-negative numeric value	₹	High
Financial summary	Income, expense, balance, savings rate	₹ / %	High
Category aggregation	Sum expenses by category	₹	High
Budget monitoring	Spent, remaining and utilization	₹ / %	Medium

3.4 Alternative Solutions & Evaluation

Criterion	Weight	C++ file-based system	Spreadsheet	General finance app
Transparency	30%	High	Medium	Medium
Cost	20%	High	High	Medium
Learning value	20%	High	Low	Low
Ease of verification	15%	High	Medium	Medium
Advanced features	15%	Low	Medium	High

3.5 Selected Design & Detailed Design

The selected design uses a simple transaction structure and a vector of transactions as the inmemory representation. The loadData() function reads existing records from expenses.csv, while saveData() writes the current transaction collection back to the file. The financialSummary() function calculates income, expense, balance and savings rate. The categorySummary() function groups expense values by category and produces category-wise totals.

Web / Dashboard Layer	Financial Analysis Layer	Persistent Data
Personal Finance Dashboard	Income and expense accumulation	expenses.csv
Transaction entry and monitoring	Balance and savings-rate calculation	Date / Type / Category / Description / Amount
Analytics view	Category-wise expense aggregation	Transaction vector in C++

Automated Personal Expenditure Monitoring System - Architecture

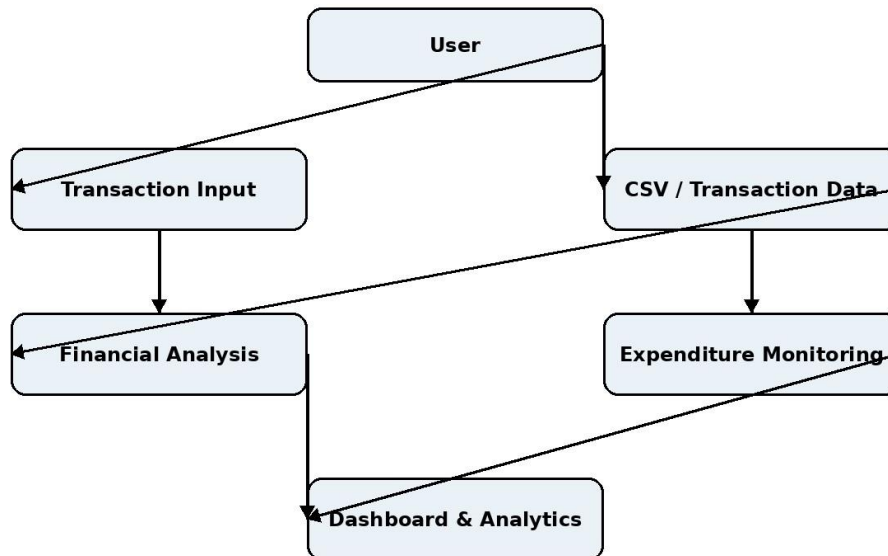


FIGURE 1. SYSTEM ARCHITECTURE AND FINANCIAL-ANALYSIS MODULE FLOW.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
C++ structures and vector	Database ORM model	Simple and course-relevant	Limited scalability	Selected for transparent academic implementation.
CSV storage	Relational database	Easy to inspect and portable	Limited concurrent access	Selected for lightweight project scope.
Direct calculations	External analytics library	Traceable and easy to verify	Fewer advanced functions	Selected because required indicators are simple.
Category vector aggregation	Map/database grouping	Demonstrates fundamental C++ concepts	Linear search for categories	Selected to keep implementation aligned with OOP/C++ learning.

3.7 Sustainability, Ethics, Safety, Risk & Security

The solution is lightweight and uses local CSV storage, which reduces infrastructure requirements and makes the system suitable for academic use. Ethical handling of financial information requires that transaction data be treated as private and that unnecessary personal details not be collected. From a safety perspective, incorrect financial calculations could mislead users, so the analysis functions must be verified against known transaction values. The implementation therefore emphasizes deterministic calculations and validation through test cases.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Use lightweight software and local storage	C++ standard library and CSV	Reduced infrastructure requirements.
Ethics	Protect financial information	Only required transaction fields collected	Avoid unnecessary personal data.
Safety	Avoid misleading financial results	Manual verification of calculated values	Testing required before interpretation.
Security	Limit data exposure	Local file storage and controlled access environment	Suitable for academic prototype; stronger security needed for production.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect transaction type	Medium	High	High	Clear type selection and testing	Low
Invalid amount	Medium	Medium	Medium	Input validation and numeric checks	Low
Incorrect category aggregation	Medium	Medium	Medium	Compare category totals with expense total	Low
CSV file failure	Low	High	Medium	Check file open/read/write operations	Low
Incorrect savings rate	Low	High	Medium	Manual formula verification	Low
Financial data exposure	Low	High	Medium	Use controlled/local test data	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development was carried out incrementally. The transaction data model and file-handling functions were established first, followed by transaction entry and display. The financial-analysis functions were then implemented and checked using known transaction values. Finally, the analysis was connected to the interactive dashboard so that calculated indicators and expenditure categories could be visually monitored.

The main resources used for the implementation were C++ development tools, the standard C++ library, CSV file storage and the Streamlit-based dashboard environment used to present the completed financial-monitoring system. The deployed application was used as practical evidence for verifying the results.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
C++	Core transaction and financial-analysis implementation	Implementation
C++ Standard Library	Vectors, file streams, string streams, formatting	Implementation
CSV	Persistent transaction storage	Implementation
Streamlit	Interactive dashboard and visualization	Presentation / Verification
Web Browser	Access deployed dashboard	Testing
Microsoft Word	Report preparation	Documentation

Implementation Modules

The financial-analysis implementation consists of the transaction data structure, persistent data loading and saving, transaction processing, financial summary calculation and category-wise expenditure aggregation. The key analytical logic is implemented in `financialSummary()` and `categorySummary()`. The first function accumulates income and expense values and derives balance and savings rate. The second function identifies unique expense categories and accumulates amounts belonging to each category.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Income total	Enter known income records and compare with manual sum	Calculated income equals expected sum.
Expense total	Sum all expense records manually and compare	Calculated expense equals expected sum.
Balance	Compute income minus expense manually	Displayed balance equals expected value.
Savings rate	Apply $\text{balance/income} \times 100$ manually	Displayed rate matches expected percentage.
Category total	Sum records within each category	Category totals equal manual aggregation.
Budget utilization	Compare spent amount with budget	Displayed utilization and remaining budget are correct.
Persistence	Save and reload transaction data	Previously stored records are available.

4.4 Results

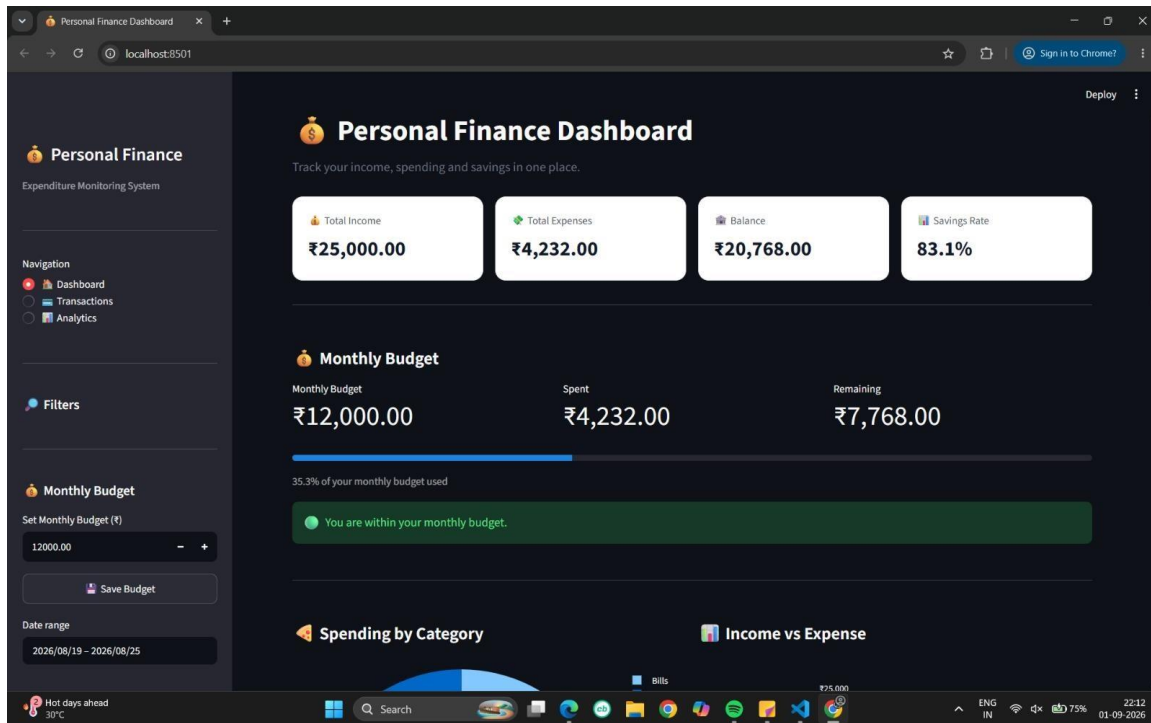
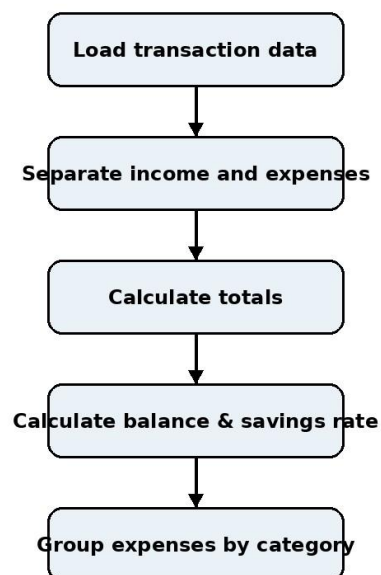


FIGURE 2 - PERSONAL FINANCE DASHBOARD SHOWING FINANCIAL SUMMARY AND BUDGET MONITORING

Financial Analysis and Expenditure Monitoring Flow



The implemented analysis was verified using the transaction records shown in the completed system. The recorded income was ₹25,000.00. Expense records totalled ₹4,232.00. Therefore, the balance was ₹20,768.00. The savings rate was calculated as $(20,768 / 25,000) \times 100 = 83.072\%$, displayed as approximately 83.1%.

FIGURE 3 - ADD NEW TRANSACTION INTERFACE

Category-wise expenditure analysis produced Bills = ₹3,652.00, Food = ₹360.00 and Travel = ₹220.00. The total of these expense categories is ₹4,232.00, matching the total expense value. The dashboard therefore identifies Bills as the highest spending category. The monthly budget was ₹12,000.00, with ₹4,232.00 spent and ₹7,768.00 remaining, corresponding to approximately 35.3% budget utilization.

Date	Type	Category	Description	Amount
19-08-2026	Income	monthly salary	salary	₹25,000.00
19-08-2026	Expense	Food	lunch	₹150.00
19-08-2026	Expense	travel	bus	₹220.00
24-08-2026	Expense	Food	snack	₹100.00
24-08-2026	Expense	Bills	recharge	₹352.00
24-08-2026	Expense	Food	breakfast	₹50.00
25-08-2026	Expense	Bills	rent	₹3,300.00
25-08-2026	Expense	Food	breakfast	₹60.00

FIGURE 4 – TRANSACTION RECORD INTERFACE

The screenshots included in this report provide evidence of the dashboard, transaction-entry interface, stored transaction records and analytics view. The results show that the financial-analysis functions are consistent with the underlying transaction data and provide useful monitoring information.

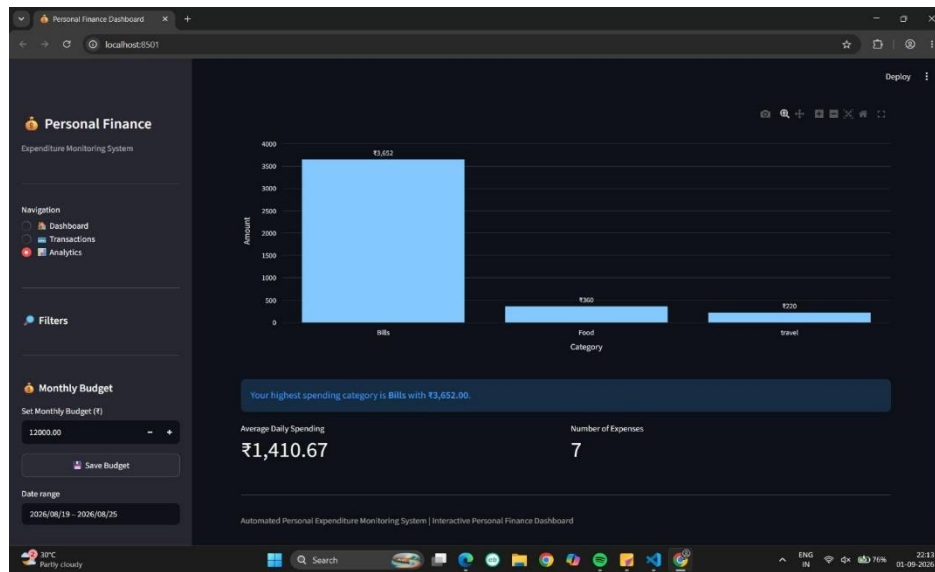


FIGURE 5 - ANALYTICS VIEW SHOWING CATEGORY-WISE EXPENDITURE

4.5 Data Analysis & Engineering Judgment

The main engineering judgment in this contribution was to keep the financial calculations simple, deterministic and directly traceable to transaction records. Instead of introducing complex financial models, the component focuses on aggregate measures that are immediately useful for expenditure monitoring. This approach reduces implementation complexity and makes verification straightforward.

The balance is computed as total income minus total expenses. The savings rate is derived from the balance relative to total income, provided income is greater than zero. Category-wise expenditure is obtained by accumulating amounts for matching expense categories. These definitions ensure that the analytical results can be reproduced manually from the transaction table.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Automated financial calculation	Financial summary displays income, expense, balance and savings rate	Fully Achieved
Expenditure monitoring	Dashboard budget section and expenditure indicators	Fully Achieved
Category-wise analysis	Analytics chart and category totals	Fully Achieved
Reliable data processing	CSV loading/saving and verified transaction records	Achieved

4.7 Strengths & Limitations

Strengths: The component further provides useful financial indicators and summaries that allow users to obtain an overall view of their financial condition. Information such as total income, total expenses, available balance, savings rate and category-wise spending helps users make better decisions regarding their expenditure. The integration with an interactive Streamlit dashboard provides a userfriendly visual representation of these analytical results. Charts, summary indicators and expenditure information make it easier for users to interpret financial data than relying only on raw transaction records.

Limitations: Despite these strengths, the system has certain limitations. The use of CSV-based storage is simple and suitable for a personal-scale application, but it is less robust than a relational or cloud database when handling a very large number of transactions. CSV files may also provide limited support for concurrent access, structured querying, advanced data management and strong data integrity mechanisms. Therefore, the current storage approach may require improvement if the system is expanded for a larger number of users or a significantly larger dataset.

4.8 Project Planning, Roles & Budget

The project was planned and developed in different stages, beginning with requirement identification and system design, followed by transaction management, financial analysis, data storage, dashboard development, testing, integration and documentation.

The development was organized into separate modules so that each team member could focus on a specific part of the system. This modular approach helped in completing the project systematically and made it easier to test and integrate the individual components.

The responsibilities were divided among the three team members. M. Sasidhar (192425304) worked on Part 1 – Income & Expenses Management, which includes recording income and expense transactions and maintaining the transaction data.

P. Muni Karthik (192425061) worked on Part 2 – Financial Analysis & Expenditure Monitoring, which includes financial calculations, expenditure aggregation, balance calculation, savings-rate calculation, category-wise expense analysis and budget monitoring. P. Karthekeya (192411093) worked on Part 3 – Dashboard, Visualization & System Integration, which includes dashboard development, graphical representation of financial information and integration of the system components.

For my individual contribution, the work was planned around the processing and analysis of the transaction records provided by the system. The financial-analysis functions were implemented to calculate total income, total expenses, balance and savings rate

. Expense records were further grouped according to their categories to determine category-wise expenditure. The budget-related information was also considered for monitoring the amount spent against the predefined monthly budget. These activities were performed as part of the financial-analysis and expenditure-monitoring layer of the project.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The financial-analysis and expenditure-monitoring component successfully converts stored transaction data into useful financial indicators. The implemented functions calculate total income, total expenses, balance, savings rate and category-wise expenditure, while the dashboard provides a practical monitoring view of the resulting information.

For the tested transaction set, the system produced ₹25,000.00 total income, ₹4,232.00 total expenses, ₹20,768.00 balance and approximately 83.1% savings rate. Category aggregation produced Bills as the highest spending category with ₹3,652.00. These results were consistent with manual verification of the transaction records.

The work demonstrates how fundamental C++ programming constructs can be applied to a practical personal-finance problem. The component is simple enough to understand and test while still providing meaningful expenditure-monitoring functionality.

5.2 Future Work

Future improvements can include database-based storage, stronger input validation, exportable financial reports, recurring-expense handling, date-range comparison, monthly trend analysis and more detailed budget alerts. Additional analytical functions could calculate category percentages, month-to-month changes and spending trends.

Future versions may also incorporate secure user authentication, encrypted storage, richer visualizations, mobile-friendly access and optional predictive analysis. These extensions would improve practical usability while requiring additional attention to security, privacy and data-management requirements.

5.3 Professional Learning & Individual Reflection

Student	Reflection
P. Muni Karthik	Learned how transaction data can be transformed into meaningful financial indicators using C++. Strengthened understanding of vectors, file handling, string streams, loops, conditional logic and formatted output. Improved ability to verify computed results against manually derived values and to interpret expenditure patterns for practical monitoring.

References

- [1] Bjarne Stroustrup, “The C++ Programming Language,” Addison-Wesley.
- [2] C++ Reference, “C++ Language and Standard Library Reference.”
- [3] Streamlit Documentation, “Streamlit API and Application Development Documentation.”
- [4] ISO/IEC 14882, “Programming Languages — C++.”
- [5] Project implementation and transaction-analysis test data, Automated Personal Expenditure Monitoring System, 2026.
- [6] Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, *C++ Primer*, Addison-Wesley Professional.
- [7] Herbert Schildt, *The Complete Reference: C++*, McGraw-Hill Education.
- [8] Nicolai M. Josuttis, *The C++ Standard Library: A Tutorial and Reference*, Addison-Wesley.
- [9] Bjarne Stroustrup, *Programming: Principles and Practice Using C++*, Addison-Wesley Professional.
- [10] W3Schools, “C++ Programming Tutorial and Reference.”

APPENDICES

Appendix A – Project Architecture / Evidence

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <vector>
#include <iomanip>
using namespace std;

struct Transaction {
    string date;    string
    type;    string
    category;    string
    description;
    double amount;
};

vector<Transaction> transactions;

void loadData() {    ifstream
    file("expenses.csv");

    if (!file.is_open())
        return;
```

```

    string line;
getline(file, line); while
(getline(file, line)) {
stringstream ss(line);
Transaction t; string
amount;

    getline(ss, t.date, ',');
getline(ss, t.type, ',');
getline(ss, t.category, ',');
getline(ss, t.description, ',');
getline(ss, amount, ',');

    if (!amount.empty()) {

        t.amount = stod(amount);
transactions.push_back(t);
    }

}

file.close();

}

void saveData() { ofstream
file("expenses.csv");

    file << "Date,Type,Category,Description,Amount\n";

```

```

    for (const auto& t : transactions) {
file << t.date << ","          <<
t.type << ","
        << t.category << ","
        << t.description << ","
        << fixed << setprecision(2)
        << t.amount << "\n";

    }

```

```

    file.close();
}

```

```

void addTransaction() {
Transaction t;      int
choice;

```

```

    cout << "\n1. Income\n";
cout << "2. Expense\n";
cout << "Enter type: ";
cin >> choice;
cin.ignore();

```

```

t.type = (choice == 1) ? "Income" : "Expense";

```

```

cout << "Enter date (DD-MM-YYYY): ";
getline(cin, t.date);

```

```
    cout << "Enter category: ";
    getline(cin, t.category);
```

```
    cout << "Enter description: ";
    getline(cin, t.description);
```

```
    cout << "Enter amount: ";
    cin >> t.amount;
```

```
    transactions.push_back(t);
```

```
    saveData();
```

```
    cout << "\nTransaction added successfully!\n";
```

```
}
```

```
void viewTransactions() {    if
(transactions.empty()) {    cout << "\nNo
transactions available.\n";    return;
}
```

```
    cout << "\n";
    cout << left
        << setw(15) << "Date"
        << setw(12) << "Type"
        << setw(15) << "Category"
        << setw(25) << "Description"
        << setw(12) << "Amount" << "\n";
```

```

cout << string(79, '-') << "\n";

for (const auto& t : transactions) {
cout << left
    << setw(15) << t.date
    << setw(12) << t.type
    << setw(15) << t.category
    << setw(25) << t.description
    << setw(12) << fixed << setprecision(2) << t.amount
    << "\n";
}
}

```

```

void financialSummary() {
double income = 0;
double expense = 0;

```

```

    for (const auto& t : transactions) {
if (t.type == "Income")
income += t.amount;    else
expense += t.amount;
    }

```

```

double balance = income - expense;

```

```

cout << "\n===== FINANCIAL SUMMARY =====\n";

```

```

cout << fixed << setprecision(2);

    cout << "Total Income   : Rs. " << income << "\n";
cout << "Total Expense   : Rs. " << expense << "\n";
cout << "Balance         : Rs. " << balance << "\n";

    if (income > 0) {

        double savingsRate = (balance / income) * 100;
cout << "Savings Rate   : " << savingsRate << "%\n";
    }

    cout << "===== \n";
}

void categorySummary() {
if (transactions.empty()) {
cout << "\nNo expense data
available.\n";    return;
}

    vector<string>          categories;
vector<double> amounts;

    for (const auto& t : transactions) {
if (t.type != "Expense")
continue;

        int index = -1;

```



```

        for (int i = 0; i < categories.size(); i++) {
if (categories[i] == t.category) {
index = i;          break;
        }
    }

    if (index == -1) {
categories.push_back(t.category);
amounts.push_back(t.amount);
    } else {

        amounts[index] += t.amount;

    }
}

cout << "\n===== CATEGORY-WISE EXPENSES =====\n";

for (int i = 0; i < categories.size(); i++) {

    cout << left << setw(20)

        << categories[i]

        << "Rs. " << fixed << setprecision(2)

        << amounts[i] << "\n";

}

cout << "===== \n";

}

```

```

int      main()      {
loadData();

    int choice;

    do {      cout <<
"\n\n";
        cout << "=====\\n";
cout << " AUTOMATED PERSONAL EXPENDITURE SYSTEM\\n";
cout << "=====\\n";
cout << "1. Add Income/Expense\\n";      cout << "2. View
Transactions\\n";      cout << "3. Financial Summary\\n";      cout << "4.
Category-wise Expenses\\n";      cout << "5. Exit\\n";
        cout << "=====\\n";
cout << "Enter your choice: ";      cin >> choice;

        switch  (choice)  {
case 1:
            addTransaction();

            break;

        case 2:
viewTransactions();
break;
            case
3:
            financialSummary();
break;
            case
4:

```

```

        categorySummary();
break;
    case
5:
        cout << "\nThank you!\n";
break;

default:
        cout << "\nInvalid choice!\n";

    }

} while (choice != 5);

return 0;
}

```

Appendix B – Project Resource Links

Streamlit Application: <https://expendituremonitoringsystem-9w2kfmbi6buug4pkrxjnta.streamlit.app/>

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence / SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.7)
Project planning and management	SO5	Ch. 3 (3.5), Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)